

Lightweight confidential and traceable multi-stakeholder data processing

Steylor Barros¹[?????], Eduardo Falcão²[0000-zzz-zzz-zzzz], and Andrey Brito¹[0000-0002-7350-8599]

¹ Universidade Federal de Campina Grande, Campina Grande/PB, Brazil

² Universidade Federal do Rio Grande do Norte, Natal/RN, Brazil

steylor.barros@ccc.ufcg.edu.br

eduardo@dca.ufrn.br

andrey@computacao.ufcg.edu.br

<https://www.lsd.ufcg.edu.br/>

Abstract. We leverage technologies such as confidential computing and distributed ledgers to propose an architecture for multi-stakeholder, confidential data processing. Our approach aims to protect non-interactive data-processing applications in scenarios where multiple parties (e.g., data controllers, application developers, and infrastructure managers) distrust one another. We generalize previous results, describe the architecture and requirements of the involved components, and present an implementation focused on lightweight execution in the AWS cloud. Our implementation focuses on Python applications, showing overheads as low as $X\%$.

Keywords: Multi-stakeholder data processing · Privilege separation · Lightweight virtualization (microVMs) · Confidential computing · Distributed ledger · Traceability · Supply-chain security

1 Introduction

Confidential data marketplaces must simultaneously support asset exchange, access control, and third-party computation over sensitive inputs—a combination that creates a difficult systems problem. A marketplace that accepts externally supplied applications cannot treat code execution as an isolated implementation detail, because the same platform also handles persistent metadata, payment and access logic, storage backends, and the datasets that motivate the marketplace in the first place. The problem is therefore not merely how to “run code safely,” but how to prevent build-time and run-time compromise from expanding into a full platform compromise.

The most direct implementation path would be a centralized confidential virtual machine [1] hosting the marketplace-facing logic, local artifact storage, and the execution orchestrator in one trust domain. That design minimizes deployment complexity and avoids inter-service coordination, but it creates the wrong

security shape for the problem: the same node that validates access and manages persistent sensitive state also becomes the place where untrusted or semi-trusted workloads are prepared and executed.

This paper argues that such a concentration of privilege is fundamentally ill-suited for high-assurance marketplaces, and proposes an architecture designed under the assumption that component compromise is an eventual certainty rather than a remote possibility. By decomposing the marketplace into distinct trust domains—control, build, and execution [5]—a failure or malicious exploit in one phase cannot propagate to the whole platform. The focus is on minimizing the “blast radius” of errors and attacks: even if the build or execution planes are corrupted, the core sensitive assets and persistent state remain protected.

The paper proceeds as follows. Section 2 introduces the three building blocks the architecture rests on—lightweight isolation, distributed ledgers, and distributed storage—each by the property it contributes, and Section 3 defines the problem and derives the requirements R1–R6 a secure solution must satisfy. Section 4 presents the three-plane architecture, its threat model, and how it generalizes the SGX-based DNAT; Section 5 describes the prototype that realizes the planes. Section 6 evaluates the design for security (a STRIDE analysis mapped to reproducible tests) and cost (a live A/B comparison against a centralized baseline). Section 7 positions the work against related trusted-execution and ledger-based systems, and Section 8 concludes.

2 Background

The architecture in this paper combines three building blocks, each introduced here by the property it contributes rather than by its internal mechanics: lightweight isolation for running partially trusted code in separate trust domains, a distributed ledger for tamper-evident traceability of who is allowed to do what, and a distributed store for the large artifacts that the ledger cannot hold economically.

2.1 Lightweight isolation with microVMs

The platform must execute application code and resolve third-party dependencies—both only partially trusted—without letting a compromise spread across trust domains. Operating-system containers share the host kernel and offer a comparatively weak boundary for this purpose, whereas full virtual machines impose a startup and memory cost hard to justify when isolation is required per task. MicroVMs such as Firecracker [2] occupy the middle ground: each workload runs behind a virtualization boundary with its own kernel, but with a minimal device model that keeps startup time and memory footprint low. Two levers thus become practical at the granularity of a single execution. *Ephemerality*: a fresh microVM per task leaves no residual state for a later run to inherit. *Network confinement*: the environment that consumes the sensitive dataset can be launched with no network interface at all, removing the primary exfiltration channel even under a complete runtime compromise.

2.2 Distributed ledgers and traceability

A distributed ledger [6] provides a tamper-evident, append-only record shared among parties that do not trust one another: entries are chained by cryptographic hash, so rewriting past history is computationally infeasible without controlling most of the network. Beyond integrity, this is what yields traceability: every consequential action—registering an asset, purchasing access, granting a permission—is recorded as a transaction that no single party can later repudiate or rewrite, producing an auditable provenance trail over the lifecycle of each asset. Smart contracts [4] push this further by turning the access policy itself into an executable, publicly verifiable program: here the ledger holds the access rights and the integrity bindings—the content hashes of the registered assets—so any stakeholder can verify what was authorized and which artifact was actually used. The cost is that every write requires consensus, adding a fee and latency, which is precisely why the large assets must live off the ledger.

2.3 Distributed storage

The assets themselves—encrypted application and dataset artifacts—are far too large to place on the ledger. They require an off-chain store that is *content-addressed*, so that the identifier of an artifact is a deterministic hash of its bytes and can be bound to the integrity record kept on the ledger, and *distributed*, so that the data stays available to the participating parties without a single trusted custodian. Any storage layer meeting these two objectives is compatible with the architecture. The InterPlanetary File System (IPFS) [3] is one such option: it identifies files by content hash and locates them through a distributed hash table. We use IPFS in a deliberately simplified form—a node co-located with the control plane rather than a public swarm—as a stand-in for any backend providing content addressing and distributed availability.

3 Problem definition and solution requirements

The core challenge addressed by the proposed architecture involves a transaction between two mutually distrusting parties: a *data buyer*, who owns a confidential application, and a *dataset provider*, who holds sensitive data. The objective is to enable a computation in which the buyer’s application processes the provider’s dataset without exposing either asset to the other party or to the underlying cloud infrastructure.

A straightforward implementation would deploy a single Confidential Virtual Machine (CVM) responsible for all platform functions: receiving uploads, managing the IPFS storage layer, interacting with the Smart Contract, resolving and installing Python dependencies, and running the application against the dataset. While operationally simple, this monolithic design concentrates all privileges within one trust domain. The critical vulnerability emerges during dependency resolution: installing external packages to satisfy an application’s requirements necessarily executes arbitrary third-party code with the same privileges as

the marketplace layer, the IPFS node, and the blockchain client. A maliciously crafted package—or a compromised dependency in the supply chain—could exploit this phase to exfiltrate stored datasets, tamper with marketplace metadata, or establish a persistent backdoor affecting future executions. A build-time compromise thus propagates immediately into a full platform compromise, making dependency resolution a disproportionately dangerous entry point.

This observation reframes the core problem. Secure execution of the application at runtime is necessary but not sufficient. An equally important concern is preventing a compromise during the build and dependency resolution phase from reaching the broader platform. The problem must therefore be treated as a *privilege separation problem*: distinct operational phases—control, build, and execution—carry fundamentally different risk profiles and must occupy separate trust domains with different network-access policies, persistence rights, and failure blast radii.

Three design consequences follow from this reframing. First, the component managing marketplace state, blockchain interaction, and IPFS operations must never directly execute dependency installation logic for user-supplied applications. Second, the build phase, where external network access is unavoidable, must be isolated from both persistent sensitive storage and the execution runtime. Third, the execution phase, during which the sensitive dataset is consumed, must operate without any outbound network interface, eliminating the primary exfiltration channel even under a complete runtime compromise.

A secure solution must satisfy the following requirements.

- **R1 (Asset confidentiality).** Neither the buyer’s application nor the provider’s dataset is revealed to the other party or to the infrastructure operator.
- **R2 (Privilege separation).** No single component holds the privileges of all phases; a compromise of one plane must not extend to the assets of the others.
- **R3 (Build isolation).** Dependency resolution—which runs partially trusted third-party code with network access—must be isolated from persistent sensitive storage and from the execution runtime.
- **R4 (Execution containment).** The plane that consumes the dataset must have no outbound network and must keep no state across runs, so that a runtime compromise can neither exfiltrate nor persist.
- **R5 (Traceability and access control).** Asset registration, access grants, and artifact integrity must be recorded on a tamper-evident, auditable ledger, and execution must be gated by a verifiable access check.
- **R6 (Application generality).** The platform must run unmodified application code with arbitrary declared dependencies, without recompilation or enclave-specific adaptation.

Section 6 returns to these requirements: R1–R5 are exercised by the STRIDE analysis (Table 2) and R6 by the cross-architecture comparison (Table 3).

4 Proposal

This section presents the architecture as a response to requirements R1–R6.

4.1 Threat model and trust assumptions

The system mediates between mutually distrusting parties: a dataset provider, an application buyer, and the marketplace operator that runs the platform. We assume an adversary that can submit a malicious application, declare a malicious or compromised dependency, or take control of an individual plane at runtime; the design goal is to bound what such an adversary reaches. Three capabilities are therefore in scope: malicious code at execution time, malicious code at build time (supply chain), and a fully compromised plane.

We treat the following as trusted. The distributed ledger executes its smart contracts correctly and is tamper-evident, so that recorded access rights and content hashes are authoritative. The virtual-machine monitor and the host enforce the microVM boundary; in the lightweight model targeted by the prototype this extends to trusting the cloud operator, so the boundary being defended is the one between the planes, not the one against the operator. A production deployment removes the operator from the trusted base by running each plane inside a confidential VM (e.g., AMD SEV-SNP [1]), which adds memory encryption and remote attestation without changing the architecture or the properties evaluated in Section 6. Out of scope are micro-architectural side channels, load-induced denial of service, and—in the single-host prototype—the shared control network among planes, reported as a residual weakness in Section 6.

4.2 Architecture

The architecture composes the building blocks of Section 2 into three planes, each a separate trust domain with its own persistence rights and network policy, and each matching one of the operational phases of Section 3—control, build, and execution—so that the different risk profiles of those phases are met by different containment. Table 1 summarizes the division; the principle throughout is that a plane holds only what it needs and reaches only what it must.

The control plane (CVM1) is the only stateful, externally reachable component, and it never resolves dependencies nor runs user code: on registration it encrypts each artifact and publishes only the encrypted blob and a manifest to the distributed store, recording the artifact’s content hash and the access metadata on the ledger. The build plane (CVM2) is the only plane with general network egress, because resolving dependencies requires reaching package indexes. The execution plane (CVM3) is the most constrained, running the application over the dataset with no network interface and no persistent state.

Figure 1 traces a single end-to-end request over the numbered interactions I1–I8, which also serve as the unit of analysis for the STRIDE evaluation in Section 6. (1) The provider and the buyer register their assets (I1); the control plane encrypts each artifact, publishes the ciphertext to the store (I3), and records its

Table 1. The three planes. Each is a separate trust domain with its own persistence rights and network policy; “Holds / persists” lists the sensitive state resident in the plane.

Plane	Role	Network	Holds / persists
CVM1 Control	Marketplace key custody, client, storage node	API, Clients and ledger RPC	Encryption and wallet keys, storage repository, marketplace state
CVM2 Build	Resolve and bundle dependencies	Egress to package indexes; private ranges filtered	Dependency cache only (read-only at use)
CVM3 Execution	Run the application over the dataset	None	Nothing (ephemeral; fresh microVM per run)

content hash on the ledger (I2). (2) The buyer purchases access (I1), recorded by the contract as a grant bound to the two asset hashes and the buyer’s address (I2). (3) On an execution request the control plane first verifies that grant on chain (I2); only then does it assemble a build bundle (source plus dependency manifest) and forward it to the build plane (I4), which resolves dependencies against public package indexes through filtered egress (I5) and returns a read-only application artifact (I6). (4) The control plane provisions input and output disks, attaches the artifact read-only, and launches the networkless execution microVM (I7); the result is read back from the output disk (I8) and the working state torn down. Because each boundary crossing carries only what the next phase needs—a bundle to the builder, an artifact and disks to the executor—a compromise on either side of a boundary cannot widen into the privileges of the other.

4.3 From DNAT to a generalized architecture

The original DNAT [5] addresses the same marketplace problem but binds it to a single substrate: applications run inside SGX enclaves, a process-level TEE that constrains both the threat model and the kind of code that can run. Our contribution is to restate the problem as one of privilege separation across operational phases—independent of the isolation technology—and to instantiate it on a substrate that relaxes those constraints: a microVM (Firecracker) in the prototype, targeting a confidential VM (SEV-SNP) in production, with the enclave-centric decomposition replaced by explicit control, build, and execution planes whose blast radius is bounded per plane.

Two consequences follow. By shipping dependencies as ordinary packages inside a read-only artifact rather than recompiling or adapting them into an enclave, the platform runs unmodified applications with arbitrary declared dependencies (R6), which the enclave model does not (the reuse row of Table 3). And because the architecture is defined over roles rather than over a specific

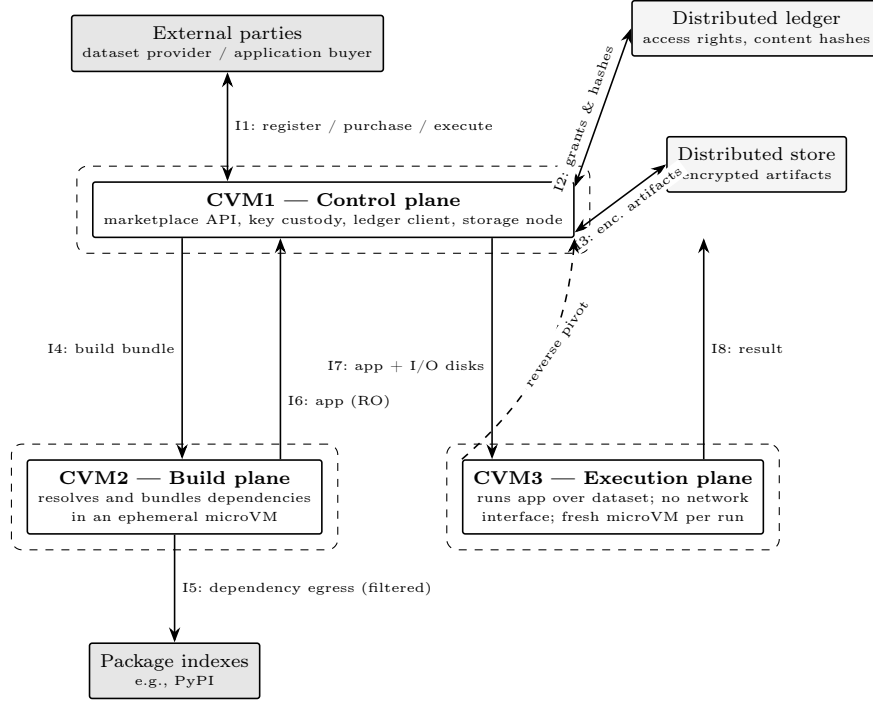


Fig. 1. The three planes and their numbered interactions (I1–I8). Dashed boxes mark trust boundaries; the STRIDE analysis of Section 6 is performed over these elements and interactions. The dashed arrow marks the reverse pivot (a compromised executor reaching the control plane), the residual quantified in Table 4.

trusted-execution technology, the same design admits different instantiations: the prototype trusts a confidential-computing root (the cloud), which is what buys the dependency freedom and the lightweight execution, whereas DNAT minimizes trust in the platform—including the operator—at the cost of that freedom; a confidential-VM deployment removes the operator from the trusted base without altering the planes or their boundaries. DNAT is thus one point in a design space that this architecture makes explicit.

5 Implementation

This section describes how the three planes are realized in the prototype, so that the security properties evaluated later can be traced to concrete mechanisms rather than to design intent alone. The system is deployed as three containers on a shared runtime—**dnat-client** (CVM1), **dnat-builder** (CVM2), and **dnat-executor** (CVM3)—each standing in for the hardware-backed confidential VM [1] of a production deployment (Section 4), and is contrasted with two baselines: a single-container deployment that colocates the same assets (A) and a

no-isolation variant that removes the microVM entirely (M0). The consequences of that substrate substitution are examined in Section 6.

5.1 Control Plane (CVM1)

The control plane hosts the marketplace API, a local IPFS node, and the blockchain client, exposing registration, access checks, and execution orchestration through a small HTTP API. On application registration, CVM1 does not install dependencies itself: it assembles a build bundle (the source file plus a manifest of declared dependencies) and forwards it to the builder. When the builder returns an `application.ext4` artifact, CVM1 encrypts it with AES-256-GCM over a gzip-compressed image, in a versioned binary envelope (magic header, length prefix, and a JSON header carrying the IV, authentication tag, and metadata). Only the encrypted blob and a manifest are published to IPFS; the plaintext artifact never leaves the control plane. Access control is enforced on chain: an execution request first calls `hasAccessByIds`, and the workflow aborts with *access denied* if the caller has not purchased access to both the dataset and the application. A purchased right is recorded under the key `keccak256(encryptedDatasetHash, encryptedApplicationHash, buyer)`—bound to the encrypted IPFS CIDs of the two assets and the buyer’s address rather than to mutable asset identifiers, so the grant is stable under off-chain re-indexing—and is verified before any bundle is assembled.

5.2 Build Plane (CVM2)

The builder exposes a single `/build` endpoint that accepts a bundle and launches an ephemeral Firecracker microVM to resolve dependencies. Unlike the executor, the build microVM is given a network interface, because resolving and compiling Python wheels may require egress to package indexes. Egress is constrained on the host side: a TAP device is created and iptables rules `REJECT` traffic from the guest toward private ranges (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 127.0.0.0/8, 169.254.0.0/16, and the host TAP address), while the remaining outbound traffic is masqueraded. Inside the guest, an artifact builder produces a read-only `ext4` image holding the application source under `/app`, its `site-packages`, a lock file of resolved dependencies, and a marker file used later for disk discovery. Dependencies are first wheel-built and then installed with `-no-index` from the local wheelhouse, and reusable wheels are persisted in a size-pruned cache that is the builder’s only durable state. After the build, the host returns only the artifact and its metadata; the per-request build state is discarded.

5.3 Execution Plane (CVM3)

The executor exposes a single `/execute` endpoint. For each request it copies a fresh root overlay, creates new input and output `ext4` disks, optionally attaches

the application artifact disk read-only, and launches a Firecracker microVM with one vCPU, 512 MiB of memory and—crucially—no network interface. The guest does not receive disk roles out of band; it discovers them by scanning candidate block devices for marker files, then extracts the workspace and runs `workspace/run.sh`. The launcher tries a small, fixed set of invocation strategies, so an application may take the dataset either as `-dataset data/dataset.csv` or as a positional argument; standard output and error are captured and persisted to the output disk. Host and guest synchronize over the serial console: the guest writes an `EXECUTION_COMPLETE` marker, after which the host issues a graceful shutdown, escalating to termination if needed, then mounts the output disk and derives the HTTP response solely from the persisted result. A `cleanup` trap unmounts every loop device and removes the entire per-run working directory on exit, which is what makes execution state ephemeral.

5.4 Application Contract and Dependency Reuse

An application is registered as unmodified source together with a manifest of its third-party dependencies. The builder (CVM2) resolves and bundles them, with the source, into the read-only `application.ext4` artifact, and the executor (CVM3) runs the application through the fixed `workspace/run.sh` entry point, passing the dataset by the conventions above. Because dependencies are shipped as ordinary `site-packages` inside the artifact—rather than linked into a trusted-execution enclave at build time—an application that follows this contract runs with arbitrary declared dependencies, unchanged across all three environments, without recompilation or enclave-specific adaptation. This is the reuse property that Section 6 contrasts with the enclave-based original DNAT, where third-party dependencies require recompilation or adaptation.

5.5 Asset Integrity and Baselines

Assets are bound to the chain by content hash: registration records the SHA-256 of the artifact, which lets a consumer detect tampering between IPFS and the contract. The first baseline (`docker/baseline-vm.compose.yaml`, environment A) runs the identical control, build, and execution logic, including Firecracker isolation for the build and execution microVMs, but inside a single container whose filesystem holds the IPFS repository, the wheel cache, and the executions store, and whose environment holds the asset-encryption key and the wallet private key. Its only structural difference from the compartmentalized deployment is the colocation of these assets—exactly the variable the security evaluation isolates.

The second baseline removes isolation altogether. With `DNAT_NO_MICROVM` set, the executor runs `vm/run-direct.sh`, which executes the same `workspace/run.sh` contract directly in the container’s namespace—no Firecracker, no per-run input and output disks, no network removal—while emitting the same `{returncode, stdout, stderr}` result, so the request path is otherwise unchanged. In this

variant (M0) the application inherits the container’s network, environment (including the control-plane secrets), and persistent filesystem, which is what lets the evaluation separate the microVM’s operational cost from the containment it provides.

6 Evaluation

The architectural argument of this paper is comparative: the compartmentalized design should contain compromise better than less-separated alternatives while paying an acceptable operational cost. We therefore evaluate it along two axes. For *security*, we organize the analysis with the STRIDE model [7], applied per interaction over the elements of Figure 1, showing how the design neutralizes—or explicitly fails to neutralize—each non-trivial threat, and mapping every claim to a concrete test. For *cost*, we measure the latency and build time of the isolation strategy.

The comparison spans up to four points on an isolation spectrum (Table 3): a no-isolation monolith (M0), a centralized deployment with ephemeral microVMs but colocated assets (Environment A), the compartmentalized design that also separates the planes (Environment B), and, as a cited reference, the SGX-based original DNAT [5]. Because A and B share the same Firecracker microVMs, any cost or security difference between them isolates the effect of *compartmentalization*; M0 isolates the cost and the security delta of the *microVM* primitive itself; and the SGX reference frames the reuse and dependency limitations that motivate our change of technology.

6.1 Methodology

We exercise the running prototype with a small suite of purpose-built applications, each designed to attack one STRIDE category, over a synthetic, PII-free dataset. Tests targeting the execution plane submit an application bundle directly to the executor, reproducing the execution contract the control plane generates, so that the variable under study—the execution microVM—is isolated from unrelated services; tests targeting lateral movement inspect the executor environment, and the access-control test drives the marketplace API. Each adverse application emits machine-readable evidence markers that a harness collects, and each test is mapped to the interaction and STRIDE category it supports (Table 2), so that every observation connects to a property of a specific boundary crossing rather than merely listing successful checks.

The suite comprises eight probes, referred to by code throughout this section so that every “ T_n ” entry in Tables 2 and 4 traces back to a concrete, reproducible adversarial application. **T1** and **T1b** drive the execution plane (I7): the first attempts outbound network exfiltration of the dataset, the second scans the environment and the disks for control-plane secrets and sibling data. **T2** declares a malicious dependency whose import-time payload runs inside the execution microVM (I4/I5), with two variants—**T2a**, which retries network exfiltration from that payload at run time (I7), and **T2b**, whose build-time

`setup.py` tries to poison the read-only wheel cache and artifact (I6/I7). **T3** repeats executions to probe the executor for residual sockets, disks, and mounts. **T4** has a compromised executor pivot toward the control plane’s secrets and stores (CVM3 $\rightarrow$ CVM1). **T5**, the only end-to-end test, drives the marketplace API (I1–I3): it requests an execution without `purchaseAccess` and audits that registration and purchase were recorded on chain and that the on-chain hash matches the artifact. The measured outcome of each probe is the corresponding row of Table 2.

Environments A and B were both measured live and end to end, on the same host as the no-isolation baseline (M0); the on-chain gas cost was measured on Hardhat. Together these complete Table 3, leaving only the SGX reference column, cited from the original DNAT work.

6.2 Security Evaluation with STRIDE

STRIDE analyzes components and their interactions, not a system taken as a whole—a claim such as “the design resists tampering” is meaningful only relative to a stated element—so we fix the level of abstraction first. The analysis is performed over the elements of Figure 1 (the external parties, the three planes, each a separate trust domain, the ledger, the store, and the package indexes) and over the numbered interactions I1–I8 that cross their trust boundaries. Following STRIDE-per-element practice [7], each of the six threat categories was examined against every element and boundary-crossing interaction; flows internal to a single trust domain are not analyzed. Every cell of that sweep falls into one of four groups: actively countered by a design mechanism, trivially handled by standard TLS or by a trust assumption of Section 4, an acknowledged residual, or not applicable. MicroVM and VMM escape at I7 is excluded by the trusted-VMM assumption of Section 4; what a plane compromise reaches despite it is captured by the reverse-pivot row of Table 2, which concentrates on the first group—the non-trivial threats the architecture must actively counter—naming for each the exact interaction where it arises, the countering mechanism, and the test that provides evidence. The other three groups are enumerated at the end of this subsection, so that the coverage can be audited rather than taken on trust.

Reading the evidence. Two points deserve emphasis beyond the per-interaction outcomes of Table 2. First, the containment of the execution plane is *structural* rather than policy-based—the seven outbound probes fail because no network interface exists, not because a firewall rule denies them—and the plane that runs untrusted code is also the smallest first-party codebase (about 365 lines, against roughly 590 in the builder and 1,500 in the secret-holding control plane), so the boundary an attacker reaches first is the least code to audit. Second, the supply-chain test (T2) is the paper’s thesis in miniature: a package seeded into the builder’s cache is resolved, bundled, and runs its payload at import time inside the execution microVM—yet finds no secrets and a blocked network, mirroring T1 and T1b, while its build-time `setup.py` variant executes

Table 2. Non-trivial STRIDE threats, per interaction of Figure 1 (S/T/R/I/D/E = spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). Trivial and assumption-covered cells are discussed in the text.

Interaction	Cat.	Threat & countering mechanism	Outcome (measured)	Test
I1	S	Caller without access rights; on-chain gate (<code>hasAccessByIds</code>) before any bundle is prepared	Run rejected, <i>access denied</i> , before code or data move	T5
I1, I2	R	Denial of a registration or purchase; recorded as ledger transactions	Both actions recorded on chain, auditable	T5
I3	T	Artifact modified between storage and use; SHA-256 content hash bound on chain	On-chain hash equals file SHA-256 content hash	T5
I4, I5	E	Malicious dependency runs at build time; ephemeral builder, filtered egress, no dataset	Payload runs in builder only—no secrets, no dataset; cache write denied	T2, T2b
I6, I7	T	Poisoned cache or modified artifact; mounted read-only at use	Build-time write denied (<code>Permission denied</code>)	T2b
I7	I	Dataset exfiltration over the network; execution microVM has no interface	7/7 outbound channels fail, no <code>eth0</code> work; execution microVM has no interface	T1, T2a
I7	I	Reads control-plane secrets or sibling data; no secrets, own disks only	Secrets absent from env; only own I/O disks visible	T1b, T4
CVM3 → CVM1	E	Executor pivots into the control plane; secrets and stores absent	Keys, IPFS, cache, executions absent (<i>residual</i> : shared net)	T4
CVM3	D	Residual state starves later per-run caps (1 vCPU/512 MiB) and teardown	runs; No socket, disk, or mount after 9+	T3

in the isolated builder and fails against the read-only cache. Malicious code inherits the containment of the plane that runs it rather than reaching the control plane.

Trivial, assumption-covered, and residual cells. The remaining cells fall into three groups. *Trivial.* In-transit tampering and disclosure on the external interaction I1 are handled by standard TLS; the ledger and store interactions (I2, I3) reduce to the trust assumptions of Section 4, and the access rights and content hashes they carry are public by construction; disclosure of the returned artifact (I6) is immaterial because the control plane commissioned it; and elevation of privilege through the marketplace API itself (I1) is an API-correctness assumption rather than a property the planes enforce. Two cells deserve a remark: information disclosure at the store (I3) is a mitigation we claim rather than assume—artifacts are encrypted before publication—and the design does not authenticate the package index (I5) beyond HTTPS, containing instead what its packages can do once resolved (the I4/I5 row of Table 2). *Acknowledged residuals and out of scope.* Three residuals are neither trivial nor fully closed, and we mark rather than claim them. First, the internal service channels (I4, I6–I8) are not mutually authenticated, so a compromised plane can impersonate an internal caller or tamper with the build bundle in transit; internal mutual TLS and network segmentation would close this network-reach residual, which reappears

in Table 4 and under Threats to Validity. Second, a malicious dependency in the builder could exfiltrate the buyer’s own application source over the filtered egress (I4/I5); the provider’s dataset never enters the build plane, so only the buyer’s own asset—not the sensitive data—is at risk. Third, the result returned to the buyer (I8) is out of scope in two senses: this work bounds what a malicious application can *reach*, not what an authorized computation *reveals* about its inputs, nor whether a compromised executor returned a faithful result; verifying the processing itself is a complementary line of work (Section 7). Denial of service beyond per-run state hygiene (e.g., request flooding at I1) and repudiation beyond the ledger-recorded actions of the I1/I2 row are likewise out of scope.

Read against the requirements of Section 3, the per-interaction analysis exercises R5 (rows I1, I1/I2, and I3), R3 (rows I4/I5 and I6), R1 and R4 (both I7 rows and the D row), and R2 (the CVM3 $\rightarrow$ CVM1 row); R6 is not a threat-facing property and is exercised instead by the cross-architecture comparison (Table 3).

6.3 Operational Cost

Table 3 consolidates the cost of the isolation strategy across architectures, drawing on two live campaigns. End-to-end execution latency is indistinguishable between A and B; its absolute value tracks host warmth—26.8 s on the 2026-06-11 A/B campaign and 25.2 s on the warmer 2026-06-23 host used here for the M0 comparison—but the $A \approx B$ equality is invariant across both campaigns. This time is dominated by the microVM life cycle—kernel boot, disk discovery, output-disk synchronization, and the guarded shutdown sequence—rather than by the application itself, whose computation over sixty rows completes in milliseconds.

What that fixed overhead buys is visible in the same measurement: the no-isolation baseline (M0) exposes a working `eth0` and succeeds at network exfiltration, finds all six control-plane secrets in its environment, and persists its writes, whereas A and B block exfiltration, expose no secret, and leave no residual state. Separating the planes (B vs. A) adds nothing on top of this microVM cost.

The on-chain cost is likewise architecture-independent. Registering an asset costs $\approx 389k$ gas (a dataset with an empty bloom filter; an application $\approx 327k$) and purchasing access $\approx 97k$ gas; a dataset’s optional on-chain bloom filter adds $\approx 185k$ gas per 256 bytes. These figures are identical for M0, A, and B because the registry is the shared control plane—isolation lives in the build and execution planes, not in the contract—so the blockchain cost of the user journey is unchanged by the isolation strategy. On a representative layer-2 deployment at 0.03 gwei with ETH at US\$3,000 they amount to about US\$0.035 to register an asset and under US\$0.01 to purchase access, so the marketplace is inexpensive to operate where such applications realistically run.

The build plane is more expensive, as expected from its heavier configuration (two vCPUs, network egress, and dependency resolution), but that cost is identical across the two designs, which share the same builder microVM (Table 3); the live A/B campaign of Section 6.4 confirms it end to end.

Table 3. Comparative cost across the isolation spectrum: a no-isolation monolith (M0), a centralized deployment that adds ephemeral microVMs but colocates all assets (A), the compartmentalized design that also separates the planes (B), and the SGX-based original DNAT as a cited, qualitative reference (its enclave latencies are not re-measured here).

Metric	M0	A	B	SGX (ref.)
Execution latency, benign app (s) [¶]	0.07	25.2	25.2	n/a
microVM boot/life-cycle overhead (s) [¶]	≈0	≈25.1	≈25.1	n/a
Execution latency, suite mean (s) [‡]	—	26.8	26.8	n/a
Build latency, no dependencies (s) [‡]	—	30.2	30.1	n/a
Build latency, requests +4 transitive (s) [‡]	—	34.9	38.2	n/a
Artifact size, no deps / requests (MiB) [‡]	—	16.0 / 19.6	16.0 / 19.6	n/a
On-chain gas, register / purchase (k) [§]	389/97	389/97	389/97	—
App reuse with deps, no recompile	yes	yes	yes	no

The microVM adds ≈25 s over M0’s ≈0.07 s, since the application’s compute over 60 rows is sub-second; that fixed cost is the price of a fresh, networkless microVM per run. A≈B by design, as both share the same Firecracker microVMs; the blast radius is where they diverge (Table 4). [¶]2026-06-23 same-host campaign (benign comparator; its three test apps spanned 25.2–25.8 s). [‡]2026-06-11 live A/B campaign ($n=3$ apps, warm host; M0 was not part of it); absolute latencies differ from the [¶] rows with host warmth, while the A≈B equality holds in both. [§]Gas is the EVM’s unit of work (a plain transfer is 21k); absolute costs and a layer-2 estimate are given in the text.

6.4 A/B Comparison: Live Measurement of Both Environments

To remove any inference from the comparison, we ran the full suite against both deployments live and end to end: the compartmentalized system (Environment B) and the centralized baseline (Environment A, a single **dnat-baseline** container colocating the same logic). Because both share the same Firecracker execution and build microVMs and the same control-plane access gate, the per-microVM guarantees are a constant—the security results of Section 6 (T1, T1b, T3, T5) reproduce byte-for-byte in both—so the architectural variable surfaces only in the blast radius.

Compartmentalization adds no measurable cost. Table 3 reports the operational cost on both paths: end-to-end execution latency is indistinguishable between A and B (mean ≈26.8 s each, over the three test apps), builds match within run-to-run variance (30.1 vs. 30.2 s without dependencies; 38.2 vs. 34.9 s pulling **requests** with its transitive tree), and the artifacts are byte-identical (16.0 and 19.6 MiB). The cost is paid by the microVM life cycle, which both designs share.

Plane separation changes only the blast radius. The single place where A and B diverge is cross-plane reach (Table 4, populated from live inspection rather than configuration). In B, the executor and the builder carry *none* of the control-plane secrets in their environment, and the sensitive stores (IPFS

repository, wheel cache, executions store) are absent from their namespace; in A, those same secrets and stores sit in the single namespace that also runs the execution logic. The supply-chain test (T2) inherits this split: a compromised dependency’s code executes, but in B inside a plane that holds neither secrets nor a route to them, whereas in A it lands beside every sensitive asset. This is the paper’s comparative thesis—*containment, not absolute security*—demonstrated with both environments measured, not one measured and one argued.

Table 4. Reach of a compromised execution plane over control-plane assets: the compartmentalized design (B) versus centralized baseline (A), from live inspection. “Isolated” = absent from the plane’s namespace; “Colocated” = same namespace as the execution logic.

Control-plane asset reachable from the execution plane	Env. B (proposed)	Env. A (baseline)
Asset-encryption key (<code>ASSET_ENCRYPTION_KEY</code> , env)	Isolated (NOT SET)	Colocated (SET)
Wallet private key (<code>PRIVATE_KEY</code> , env)	Isolated (NOT SET)	Colocated (SET)
Service URLs (<code>RPC_URL</code> , <code>IPFS_API_URL</code> , <code>BUILDER_URL</code> , <code>EXECUTOR_URL</code> , env)	Isolated (NOT SET)	Colocated (SET)
IPFS repository, wheel cache, executions store (filesystem)	Isolated (separate namespace)	Colocated (same filesystem)
IPFS API (port 5001, network)	Blocked (loopback in CVM1)	Reachable (127.0.0.1)
Control API (port 3001, network)	Reachable [†]	Reachable (127.0.0.1)
Blockchain RPC (port 8545, network)	Reachable [†]	Reachable (127.0.0.1)

[†]Residual risk in the single-host deployment: the three containers share one Docker network (`dnat-runtime`); across separate machines this is governed by inter-host network policy. Internal mutual TLS and network segmentation remain future work. The defining assets of the blast radius—secrets and persistent stores—are nonetheless isolated in B and colocated in A.

6.5 Threats to Validity

The measurements were taken on a single host with a Linux-backed container runtime; absolute latencies will vary with hardware (a colder first campaign saw a 31.4 s execution mean and 103–131 s builds, with the same $A \approx B$ relationship), but the containment and exfiltration results are structural (absence of a network interface, absence of secrets) and do not depend on timing. The network-reach residual of Section 6 bounds the blast-radius claim: in the single-host deployment the three planes share one Docker network, so a compromised executor still

reaches the control API and the blockchain RPC, even though the secrets and stores that define the blast radius remain isolated (Table 4). Finally, the DoS analysis covers only per-run state hygiene, not load-induced resource exhaustion, and the SGX column of Table 3 is qualitative—it restates the original DNAT’s reuse limitation rather than re-measuring enclave latencies here.

A more fundamental caveat concerns the execution substrate. The prototype runs each plane as a container with Firecracker microVMs rather than inside a hardware-backed confidential VM, so our measurements do not come from a genuinely confidential environment and we make no claim about confidential-computing overheads. This does not weaken the architectural argument: the properties we evaluate—privilege separation across planes, a networkless execution microVM, ephemeral teardown, and on-chain access control—belong to the architecture, not to the memory-encryption substrate. Running the same planes inside confidential VMs (e.g., AMD SEV-SNP [1]) would change the implementation and add a hardware-attestation and memory-encryption cost, but would preserve the containment and blast-radius results demonstrated here: confidential execution is a substrate substitution rather than an architectural change.

7 Related Work

A line of work uses trusted execution and distributed ledgers to control how sensitive data is used after it is handed to an untrusted consumer. The original DNAT [5] pioneered this combination for *auditable traceability*, recording data-and-application usage in a tamper-evident ledger while executing the application over the data inside an SGX enclave; it assumes, however, that the application has been pre-verified as non-malicious by the data owner or a trusted third party. Systems such as PrivacyGuard [9] similarly enforce usage contracts on chain and run the computation in an attested off-chain enclave, again taking the processing code as ratified in advance. The most recent and closest work, VDDPI [8], removes that assumption: it verifies the consumer’s processing code *mechanically*—through a mandatory template, an allow-listed function set, and static taint analysis executed by smart contracts on a permissioned ledger—so that a data provider can learn, before releasing data, whether the code discloses its inputs, and it binds the analysed code to the running enclave through the SGX measurement (MRENCLAVE).

Our work is complementary to that verification axis rather than competing with it. VDDPI answers *what* an authorized computation is allowed to reveal about its inputs—the disclosure-through-the-legitimate-result concern that we place out of scope (Section 6)—but pays for it with a restricted programming model: applications must fit the template and the allow-list, so arbitrary dependencies and unmodified code are, by construction, not supported. We instead target *how* partially trusted code is run: privilege separation across control, build, and execution planes bounds the blast radius of a build- or run-time compromise, and treats dependency resolution (which VDDPI forbids outright) as a first-class supply-chain threat, while running unmodified applications with

arbitrary declared dependencies. The two directions are composable: a code-verification stage in the spirit of VDDPI could be attached to our control plane without altering the plane boundaries, combining processing verification with the containment guarantees evaluated here.

8 Conclusion

This paper framed the proposed system as a layered containment architecture for a confidential data marketplace and tested that framing against the running prototype. The central design move—separating control, build, and execution into planes with different network and persistence properties—was evaluated not as an abstract claim of trustlessness but as a measurable reduction in blast radius relative to a centralized baseline that differs only in the colocation of assets.

The experimental results support that comparative argument. Execution-time code cannot open outbound channels, because the execution microVM is launched without a network interface: every probed channel failed with a structural network-unreachable error rather than a defeatable policy. Ephemeral state does not survive—after repeated executions the executor retained no disks, mounts, sockets, or datasets—and the assets that define the blast radius (the asset-encryption key, the wallet key, the IPFS repository, the wheel cache, and the executions store) are absent from the compromised executor in the compartmentalized design, whereas they are colocated in the baseline. Execution is gated on chain, and the isolation is bought at a fixed operational cost of roughly twenty-five seconds of microVM life cycle per run. Because both environments were measured live and end to end, this comparison rests on inspection of the running deployments rather than on inference from configuration.

The evaluation also makes the limitations concrete rather than rhetorical. In a single-host deployment the planes share one network, so a compromised executor can still reach the control API and the blockchain RPC even though the secrets and stores remain isolated; dataset confidentiality still depends on preparation and orchestration outside the encrypted-artifact path; and the system does not yet expose a remote-attestation flow that would let an external party verify the software identity of each confidential component before trusting a result. We therefore conclude that the compartmentalized design is justified as a credible high-assurance baseline for a confidential marketplace prototype: it demonstrably narrows what a single compromise can reach and what it can leave behind, while leaving a clearly bounded set of properties—service identity, dataset confidentiality, and attestation—as the next steps. Future work will close the network-segmentation gap between planes with internal mutual TLS, extend artifact-grade encryption and provenance to datasets and cached wheels, and add attestation, so that the containment demonstrated here can be verified remotely rather than assumed.

Declaration on the Use of Generative AI

During the preparation of this work, the authors used generative AI tools (large language models) to assist with drafting and editing parts of the manuscript text and with the development and debugging of the prototype implementation and evaluation code. After using these tools, the authors reviewed, verified, and edited all resulting content as needed and take full responsibility for the entire content of this paper. No AI tool is listed as an author, and no AI tool was used to generate or fabricate experimental data, results, or scientific claims.

References

1. Advanced Micro Devices, Inc.: AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. Tech. rep., Advanced Micro Devices, Inc. (2020)
2. Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., Popa, D.M.: Firecracker: Lightweight virtualization for serverless applications. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20). pp. 419–434. USENIX Association (2020)
3. Benet, J.: IPFS - content addressed, versioned, P2P file system. CoRR **abs/1407.3561** (2014), <https://arxiv.org/abs/1407.3561>
4. Buterin, V.: A next-generation smart contract and decentralized application platform. Ethereum White Paper (2014), <https://ethereum.org/en/whitepublication/>
5. Jr, J.R.N., Nunes, J.B.S., Falcão, E.L., Sampaio, L., Brito, A.: On the tracking of sensitive data and confidential executions. In: Proceedings of the 14th ACM International Conference on Distributed and Event-based Systems (DEBS '20). pp. 1–10. ACM, Virtual Event, QC, Canada (2020). <https://doi.org/10.1145/3401025.3404097>
6. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. White Paper (2008), <https://bitcoin.org/bitcoin.pdf>
7. Shostack, A.: Threat Modeling: Designing for Security. John Wiley & Sons, Indianapolis, IN (2014)
8. Tokuda, S., Kakei, S., Shiraishi, Y., Saito, S.: VDDPI: Verifiable decentralized data processing infrastructure for data usage control and confidential data processing. IEEE Transactions on Dependable and Secure Computing **23**(3), 6068–6084 (2026). <https://doi.org/10.1109/TDSC.2026.3658309>
9. Xiao, Y., Zhang, N., Li, J., Lou, W., Hou, Y.T.: PrivacyGuard: Enforcing private data usage control with blockchain and attested off-chain contract execution. In: Computer Security – ESORICS 2020. pp. 610–629. Springer (2020)